

AI-Enhanced Intrusion Detection System

Final Project Documentation

SmartInternz Externship Program — 2026

Submitted By: Prithviraj Nimbalkar

1. Project Overview

The AI-Enhanced Intrusion Detection System (IDS) is a machine learning-powered cybersecurity web application that detects and classifies network intrusions in real time. It uses a Random Forest Classifier trained on NSL-KDD dataset features to identify five categories of network traffic — Normal, DoS, Probe, R2L, and U2R attacks.

The system is built using Python Flask for the backend, Bootstrap 5 for the responsive frontend, and scikit-learn for the ML pipeline. It includes a live dashboard, a network traffic prediction form, an alert log, and a REST API endpoint for integration.

2. Objectives

- Build an AI-based system to automatically classify network traffic as normal or malicious
- Train a Random Forest model on NSL-KDD network intrusion dataset features
- Develop a user-friendly web interface with real-time prediction and dashboard
- Log all detections with timestamps, labels, and confidence scores
- Expose a REST API for real-time programmatic intrusion detection

3. Technologies Used

Category	Technology
Backend	Python 3, Flask 3.1
Machine Learning	scikit-learn (Random Forest Classifier)
Data Processing	NumPy, Pandas
Frontend	HTML5, CSS3, Bootstrap 5, Chart.js
Model Storage	Pickle (.pkl files)
Dataset	NSL-KDD (Network Intrusion Dataset)
Version Control	Git & GitHub

4. Folder Structure

```
AI-Enhanced-Intrusion-Detection-System/  Documentation/
Project_Documentation.docx  models/      ids_model.pkl      scaler.pkl
static/css/                  style.css    templates/         home.html          dashboard.html
predict.html                 alerts.html  about.html         app.py             train_model.py
requirements.txt             README.md
```

5. System Architecture

The system follows a standard MVC-style web application architecture with an additional ML inference layer:

- User sends request from browser to Flask web server
- Flask routes the request to the appropriate handler (dashboard, predict, alerts)
- For predictions, input features are scaled using StandardScaler and passed to the Random Forest model
- The model returns a class label (Normal/DoS/Probe/R2L/U2R) with probability scores
- The result is logged in the in-memory alert store and returned to the user

5.1 Data Flow Diagram

User Input (10 Features) → StandardScaler → Random Forest Model → Class Label + Confidence → Alert Log + Response

6. Machine Learning Model

6.1 Dataset — NSL-KDD

The NSL-KDD dataset is an improved version of the KDD Cup 1999 dataset used for network intrusion detection research. It contains labeled records of network connections classified as Normal or one of four attack categories.

6.2 Features Used (10)

#	Feature Name	Description
1	duration	Duration of connection in seconds
2	src_bytes	Bytes sent from source to destination
3	dst_bytes	Bytes sent from destination to source
4	land	1 if source/dest are same host, else 0
5	wrong_fragment	Number of wrong fragments
6	urgent	Number of urgent packets
7	hot	Number of hot indicators
8	num_failed_logins	Number of failed login attempts
9	logged_in	1 if successfully logged in, else 0
10	num_compromised	Number of compromised conditions

6.3 Model Training

- Algorithm: Random Forest Classifier (100 decision trees)
- Train/Test Split: 80% training, 20% testing
- Preprocessing: StandardScaler (zero mean, unit variance)
- Accuracy: ~96-100% on synthetic demo dataset

- Saved as: models/ids_model.pkl and models/scaler.pkl

7. Attack Categories

Type	Full Name	Description	Indicator
DoS	Denial of Service	Floods network/server to exhaust resources	High src_bytes, zero dst_bytes
Probe	Probing / Port Scan	Scans network to map vulnerabilities	Zero src/dst bytes, short duration
R2L	Remote to Local	Unauthorized remote access to local machine	High failed_logins (>3)
U2R	User to Root	Privilege escalation attacks	High hot indicators, 1+ compromised
Normal	Normal Traffic	Legitimate network connection	Balanced bytes, successful login

8. Application Routes

Route	Method	Description
/	GET	Home page with features and attack type overview
/dashboard	GET	Live stats dashboard with Chart.js doughnut chart
/predict	GET, POST	10-feature input form with demo buttons and result card
/alerts	GET	Full table of all logged predictions with confidence bars
/about	GET	Project details, tech stack, and API documentation
/api/predict	POST	REST API endpoint for programmatic predictions

9. REST API

9.1 Endpoint

POST /api/predict

9.2 Request Body

```
{"features": [0, 490000, 0, 0, 3, 0, 0, 0, 0, 0]}
```

9.3 Response

```
{"label": "DoS Attack", "confidence": 92.3, "is_attack": true, "timestamp": "2024-10-15T14:32:00"}
```

10. Setup & Run Instructions

Step 1: Install Dependencies

```
pip install -r requirements.txt
```

Step 2: Train ML Model

```
python train_model.py
```

Step 3: Run Flask App

```
python app.py
```

Step 4: Open Browser

```
http://localhost:5000
```

11. Conclusion

The AI-Enhanced Intrusion Detection System successfully demonstrates the application of machine learning in the field of network cybersecurity. By combining a Random Forest classifier with a Flask web interface, the project delivers a practical, accessible tool for detecting network intrusions in real time.

The system correctly identifies all five classes of network traffic (Normal, DoS, Probe, R2L, U2R) with high accuracy. The inclusion of a live dashboard, alert logging, demo simulation buttons, and a REST API makes it a complete, deployable application suitable for academic projects, internship showcases, and small-scale network monitoring.

— End of Document —